The New Scheduler

on the Block, Dedicated to Desktops

What in the world is the BFS and how does it fare against the mainline kernel's CFS? Here's a report.

After a two-year break from kernel hacking (during which time he probably was hacking Linux in private), Australian anaesthetist and part-time kernel developer, Con Kolivas, has emerged with a new scheduler. He claims it provides significantly better performance on dual- and quad-core processor-based desktops compared to the mainline kernel's Completely Fair Scheduler (CFS), developed by Ingo Molnar. (Read more about Kolivas' past involvement with kernel development and why he had quit in 2007 at *www.linuxforu.com/views/sad-case-of-dr-con*.)

Kolivas has named it the Brain Fuck Scheduler, or BFS, for short.

Why the name?

There are numerous reasons for this. According to the official FAQ, here are a few:

- Because it throws out everything that we know is good about how to design a modern scheduler in scalability.
- Because it's so ridiculously simple.
- Because it performs so ridiculously well in what it's good at, despite being that simple.
- Because it's designed in such a way that mainline would never be interested in adopting it, which is how I like it.
- Because it will make people sit up and take notice of where the problems are in the current design.
- Because it throws out the philosophy that one scheduler fits all and shows that you can do a *lot* better with a scheduler designed for a particular purpose. I don't want to use a steamroller to crack nuts.
- Because it actually means that more CPUs mean better latencies.
- Because I must be fucked in the head to be working on this again.
- I'll think of some more becauses later.

Now, before we get deep into BFS and its performance benefits on desktops, let's have some basic ideas about schedulers.

What's a scheduler?

The sole purpose of an operating system is to multi-task. The number of tasks (i.e., processes running in the foreground, as well as in the background) are always far greater than the number of CPUs in our computers. How does the CPU(s) attend to processing requests, which often come simultaneously, from a number of processes?